

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

I GOUTHAM KORNANA /192311169 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:K.GOUTHAM

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Answer:

A **Servlet** is a Java program that runs on a web server and processes client requests and generates responses.

Servlet Architecture

The architecture consists of:

Client/Browser → Web Server → Servlet Container → Servlet → Database

1. The student enters details in an HTML form.
2. The browser sends an HTTP request.
3. The web server receives the request.
4. The Servlet Container, such as Tomcat, identifies the required servlet.
5. The servlet processes the request.
6. The servlet sends an HTTP response to the browser.
7. Data can also be stored in a database using JDBC.

HTML Form

```
<form action="student" method="post">  
  Name: <input type="text" name="name"><br>  
  Email: <input type="email" name="email"><br>  
  Course: <input type="text" name="course"><br>  
  <input type="submit" value="Register">  
</form>
```

Servlet Using GET and POST

```
import java.io.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;

import jakarta.servlet.annotation.WebServlet;

@WebServlet("/student")

public class StudentServlet extends HttpServlet {

    public void init() {

        System.out.println("Servlet Initialized");

    }

    protected void doGet(HttpServletRequest request,

        HttpServletResponse response)

        throws IOException {

        String name = request.getParameter("name");

        String email = request.getParameter("email");

        String course = request.getParameter("course");

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<h2>Student Details</h2>");

        out.println("Name: " + name + "<br>");
```

```

        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws IOException {

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h2>Registration Successful</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }

    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}

```

Servlet Life Cycle

The servlet has three main life-cycle methods:

1. init()

It is called only once when the servlet is created. It is used for initialization.

2. service()

It is called for every client request. It processes the request and calls doGet() or doPost() depending on the HTTP method.

3. destroy()

It is called before the servlet is removed from the server. It is used to release resources.

Life cycle:

Loading → init() → service() → destroy()

GET and POST

GET

Data appears in URL

Suitable for retrieving data

Less suitable for sensitive information

Limited URL size

POST

Data is sent in request body

Suitable for submitting data

Supports larger data

POST is more appropriate for sensitive data because the submitted information is not placed in the URL. However, POST by itself does not provide encryption; **HTTPS should be used** to protect sensitive information.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication. Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Answer:

Session management is used to maintain information about a user across multiple HTTP requests. Since HTTP is stateless, a server needs session tracking to identify the same user.

Using HttpSession

```
import java.io.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (username.equals("admin") &&
            password.equals("1234")) {

            HttpSession session = request.getSession();

            session.setAttribute("username", username);

            response.sendRedirect("home");
        } else {
            response.getWriter().println("Invalid Login");
        }
    }
}
```

```
}
```

Accessing the Session

```
HttpSession session = request.getSession(false);

if (session != null &&
    session.getAttribute("username") != null) {

    String username =
        (String) session.getAttribute("username");

    response.getWriter().println("Welcome " + username);
} else {
    response.getWriter().println("Please Login");
}
```

Logout

```
HttpSession session = request.getSession(false);

if (session != null) {
    session.invalidate();
}
```

```
response.sendRedirect("login.html");
```

Using Cookies

A cookie stores small information in the user's browser.

```
Cookie cookie = new Cookie("username", "admin");
cookie.setMaxAge(3600);
```

```
response.addCookie(cookie);
```

To read a cookie:

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies != null) {  
    for (Cookie c : cookies) {  
        if (c.getName().equals("username")) {  
            System.out.println(c.getValue());  
        }  
    }  
}
```

Session Tracking

Session tracking works as follows:

Login → Server creates session → Session ID generated → Browser receives session ID → Browser sends ID with future requests → Server identifies the user

A common session ID is stored in a cookie named **JSESSIONID**.

HttpSession vs Cookies

HttpSession

Cookies

Data is mainly maintained on server Data is stored on client

Suitable for login sessions

Suitable for small preferences/identifiers

Can store objects

Stores small text values

Can be invalidated by server

Has an expiration time

Better for session information

Should not store passwords

Conclusion: HttpSession is preferred for maintaining authenticated user sessions, while cookies are useful for storing small client-side information and session identifiers.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application. Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Answer:

JDBC (Java Database Connectivity) is an API that allows Java applications to communicate with databases such as MySQL.

JDBC Architecture

The JDBC architecture is:

Java Application → JDBC API → DriverManager → JDBC Driver → Database

The important JDBC components are:

1. **DriverManager** – manages database drivers.
2. **Connection** – establishes a database connection.
3. **Statement/PreparedStatement** – executes SQL commands.
4. **ResultSet** – stores results of SELECT queries.
5. **SQLException** – handles database errors.

Database Table

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE product (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    price DOUBLE,
```

```
    quantity INT  
);
```

JDBC Program

```
import java.sql.*;
```

```
public class ProductJDBC {
```

```
    static final String URL =
```

```
        "jdbc:mysql://localhost:3306/ecommerce";
```

```
    static final String USER = "root";
```

```
    static final String PASSWORD = "root";
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```
            Connection con =
```

```
                DriverManager.getConnection(
```

```
                    URL, USER, PASSWORD);
```

```
            String sql =
```

```
                "INSERT INTO product(name, price, quantity) " +
```

```
                "VALUES (?, ?, ?)";
```

```
PreparedStatement ps =
    con.prepareStatement(sql);

ps.setString(1, "Laptop");
ps.setDouble(2, 55000);
ps.setInt(3, 10);

ps.executeUpdate();

System.out.println("Product inserted");

Statement st = con.createStatement();

ResultSet rs =
    st.executeQuery("SELECT * FROM product");

while (rs.next()) {
    System.out.println(
        rs.getInt("id") + " " +
        rs.getString("name") + " " +
        rs.getDouble("price") + " " +
        rs.getInt("quantity")
    );
}

rs.close();
```

```

        st.close();

        ps.close();

        con.close();

    } catch (ClassNotFoundException e) {

        System.out.println("Driver not found");

    } catch (SQLException e) {

        System.out.println(

            "Database Error: " + e.getMessage());

        }

    }

}

```

Steps in JDBC Connectivity

1. Load JDBC Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

2. Establish Connection

```
Connection con =
```

```
    DriverManager.getConnection(URL, USER, PASSWORD);
```

3. Create Statement

```
PreparedStatement ps =
```

```
    con.prepareStatement(sql);
```

4. Execute SQL Query

```
ps.executeUpdate();
```

For retrieving data:

```
ResultSet rs =
```

```
st.executeQuery("SELECT * FROM product");
```

5. Process Result

```
while (rs.next()) {  
    System.out.println(rs.getString("name"));  
}
```

6. Close Resources

```
rs.close();  
ps.close();  
con.close();
```

Exception Handling

JDBC operations may generate SQLException. Therefore, database operations should be placed inside a try-catch block.

```
try {  
    // Database operations  
}  
  
catch (SQLException e) {  
    System.out.println(e.getMessage());  
}
```

Using **PreparedStatement** is recommended because it safely handles input parameters and helps prevent **SQL injection**.

JDBC Flow

Web Form → Servlet → JDBC → Database → ResultSet → Servlet → Web Browser

Thus, JDBC provides a standard and efficient way for a Java web application to insert, retrieve, update, and delete product information from a database.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

